

한국어 챗봇 Q→A 응답생성 프로젝트 — 진행 로그

작성일: 2026-09-19 · 이 문서는 2026-09-19 세션의 작업 기록이다. 2026-09-18까지의 누적 기록은 **PROGRESS_LOG.pdf**를 참고한다.

0. 이번 세션 실행 환경 (이전 기록과 다름)

이전 로그는 RTX 5060 Laptop(VRAM 8GB) 기준으로 작성되었으나, 이번 세션은 다른 장비에서 진행했다. 노트북 주식 중 GPU를 언급한 부분도 이번 장비 기준으로 수정했다.

항목	내용
GPU	NVIDIA GeForce RTX 4070 Ti
Python	3.11 (.venv, 이번 세션에서 새로 생성)
PyTorch	2.11.0+cu128 (CUDA 12.8) — torch.cuda.is_available() = True 확인
Java	Eclipse Temurin JDK 17 신규 설치 (KoNLPy Okt의 JVM 의존성 해결)

1. 환경 구성

- .venv 생성 및 패키지 설치: requirements.txt는 uv 기준으로 작성되어 있으나 uv가 설치돼 있지 않아 표준 venv + pip로 동일하게 구성했다.
- requirements.txt의 dataclasses==0.8 제외: Python 3.6 이하용 백포트라 3.11에 설치 불가하며, 3.11에는 표준 라이브러리로 내장돼 있어 불필요하다. requirements.txt 파일 자체는 수정하지 않고 설치 시에만 제외했다.
- JDK 17 설치: KoNLPy는 import는 되지만 Okt() 실행에 JVM이 필요해 Eclipse Temurin JDK 17을 설치했다. 형태소 분석 정상 동작 확인(['안녕하세요', '반갑습니다']).
- Jupyter 커널 등록: nbconvert가 미설치 상태라 requirements.txt에 이미 있는 nbclient/nbformat으로 노트북을 실행했다. 커널 **nlp-project-venv**를 등록했다.
- 누락 폴더 생성: 코드가 참조하지만 없던 **dataset/processed/**, **checkpoints/**를 생성했다. 특히 06번은 checkpoints/ 생성 코드 없이 torch.save를 호출해 단독 실행 시 실패할 수 있었다.
- CLAUDE.md 수정: 데이터 경로 표기를 실제 파일에 맞춰 ChatbotData.csv → ChatbotData.xlsx로 고쳤다.

2. 1차 전체 실행 (01~06번)

01번부터 06번까지 순차 실행했다. 01번 최초 실행은 작업 디렉토리 문제로 실패해, 커널의 실행 경로를 notebooks/로 지정하도록 고쳐 해결했다(노트북들이 ../dataset 같은 상대경로를 사용).

노트북	결과
01_data_eda	원본 11,823건 → 정제 11,750건, 8:1:1 분할(9,400/1,175/1,175)
02_tokenization_embedding	BPE vocab 8,000 학습, Word2Vec 6,048단어 학습
03_baseline_transformer	30 epoch 예산 소진, val_loss 6.3724, 반복 붕괴 관찰
04_finetune_pretrained	베이스라인 117 epoch 조기종료(5.7957), KoBART 7 epoch(2.8073)
05_decoding_strategy	greedy/beam/sampling 비교 — KoBART에 구 단위 반복 잔존
06_hparam_comparison	A0/A1/A2, KoBART B1/B2, KE-T5 학습 완료

3. 07·08번 신규 작성 및 실행

- 07_evaluation.ipynb: 모델 9종 × test 1,175건 전량 정량 평가. BLEU/ROUGE-L(Okt 형태소 기준)/Distinct-1,2/빈 응답률/연속 반복률, 디코딩 전략 전체 검증, 감정 label별 분해.
- 08_error_analysis.ipynb: 오류를 자동 탐지 가능한 결함(A층)과 수동 판정 대상(B층)으로 분리해 분류, 개선 단계별 추적, 대표 실패 사례 추출.

- 평가 결과를 **results/**에 CSV·JSON·PNG로 저장해 노트북과 보고서에서 재사용하도록 했다.

4. 핵심 발견 — 전이학습 파이프라인의 EOS 누락 결함

07번 1차 평가에서 사전 예상과 반대 결과가 나왔다: 처음부터 학습한 베이스라인(BLEU 4.87 / ROUGE-L 22.99)이 사전학습 KoBART(3.83 / 17.54)를 앞섰다. "챗봇은 정답이 여러 개라 지표의 한계"로 넘기지 않고 08번에서 원인을 추적한 결과, 파이프라인 결함을 발견했다.

단계	확인된 사실
증상	KoBART 응답의 정상 문장 종결률 32.3% (베이스라인 99.6%), 평균 길이가 정답의 약 2배
직접 원인	모델이 EOS를 생성하지 않아 max_length=16까지 계속 생성하다 문장 중간에서 잘림
근본 원인	gogamza/kobart-base-v2 토큰라이저는 text_target= 인코딩 시 EOS를 붙이지 않는다. 04/06번이 이를 보정하지 않아 모델이 "문장을 끝내는 법"을 학습하지 못했다
대안 가설 기각	"타깃이 max_length에서 잘려 EOS 소실" → 실제 잘리는 타깃은 0.8%(77/9,400)뿐
베이스라인 대비	03번 encode_text()는 [BOS]+ids+[EOS]로 EOS를 직접 붙였음. 전이학습 쪽에만 빠진 처리

5. 결함 수정 및 재실행 (사용자 요청)

- 04/06번 수정: collate 함수에 **encode_targets_with_eos()**를 추가해 타깃 끝에 EOS를 붙이도록 했다. KE-T5 토큰라이저는 EOS를 자동으로 붙이므로 없을 때만 추가하도록 조건을 걸었다.
- 06번 추가 수정: 체크포인트에 토큰라이저도 함께 저장(기존에는 모델만 저장), GPU 표기를 실제 장비로 정정.
- 07번 수정: 토큰라이저 로드 경로 단순화, EOS 수정 전/후 비교 절 추가.
- 수정 전 결과 보존: 비교를 위해 1차 실행 결과를 **results/before_eos_fix/**에 백업했다.
- 재실행: 04 → 05 → 06 → 07 → 08 순차 재실행. 다른 조건은 전부 고정했으므로 "EOS 하나만 바꿨을 때의 효과"를 보는 통제 실험이 된다.

수정 효과 (04-05번 확인 시점)

항목	수정 전	수정 후
KoBART best val_loss	2.8073	2.7424
생성 예시 ("청첩장을 찍는 날이 오다니")	축하해주요! 예쁘게 찍어주세요. 예쁘게 입으세요. (잘림)	축하드려요. (완결)
05번 KoBART 연속 반복률 (8샘플)	greedy/beam/beam+norepeat 모두 0.125	세 전략 모두 0.000
베이스라인 재학습 (대조군)	117 epoch / 5.7957	117 epoch / 5.7957 (동일 재현)

베이스라인은 EOS 수정과 무관한 별도 파이프라인이므로 수치가 완전히 동일하게 재현됐다 — 통제가 지켜졌음을 확인하는 대조군 역할을 한다. 또한 05번에서 KoBART의 반복이 디코딩 전략과 무관하게 사라져, 1차 실행에서 내렸던 "no_repeat_ngram이 반복 억제에 결정적"이라는 결론이 사실은 학습 결함을 덮는 임시방편이었음이 드러났다.

6. 이번 세션 기준으로 통일한 사항

- 03~05번 요약 셀을 이번 세션 실행 수치로 교체했다(이전 실행 기준 수치가 남아 있었음).
- 이전 실행의 생성 예시 "사람 사람 사람"을 이번 세션 실제 출력인 "마음을 마음을 마음을"로 교체했다.
- 06번에 하드코딩돼 있던 04번 참조값(2.7939 → 2.7424)과 "114 epoch" 표기(→ 117 epoch)를 정정했다.

7. 남은 작업

- PLAN.md / PROJECT_FLOW.md / report/REPORT_TEMPLATE.md가 이전 실행 기준이라 갱신 필요 (사용자 확인 대기).

- 발표자료(slides/) 작성.
- label 1.2 성능 보강, 사람 평가 또는 의미 기반 지표 추가 검토.